Lab 12 — Enterprise Incident Investigation Capstone

ID: K8S-LAB-12 · **Category:** Investigation · **Level:** Enterprise · **Duration:** 120 min

Focus: P1 incident response, root cause, recovery, postmortem, blast radius

Objective

Run an end-to-end P1 investigation: alert → triage → root cause → recovery → validation → postmortem note.

Concepts Covered

• Investigation playbooks

• Blast radius reasoning

• Recovery vs root cause (fix forward vs rollback)

• Communicating during a live incident

Prerequisites

• All previous labs

Architecture

Real incidents are not a single command. They're a loop you exit only when metrics, logs and customer reports all agree the issue is gone.

```
ALERT  ->  Triage  ->  Hypothesis  ->  Test  ->  Recover  ->  Validate  ->  Postmortem
           (5-min)  (15-min)         (loop)       (act)      (prove)      (next day)
```

Lab Steps

01. Pick an incident from the ops console

Choose any K8S-INC-30xx. We'll walk through K8S-INC-3001 (CrashLoopBackOff cascade).

```
# Open /ops/k8s-incidents in the ShadowXLab app.
```

Expected: Incident detail page open.

Internals: Incidents in the lab are deterministic — symptoms map 1:1 to a known root cause.

02. Triage (5 minutes)

Three commands that give you the cluster-wide pulse.

```
kubectl get pods -A --field-selector=status.phase!=Running | head
kubectl top pods -A | sort -k4 -h | tail
kubectl get events -A --sort-by=.lastTimestamp | tail -20
```

Expected: List of unhealthy pods + top consumers + recent events.

Internals: These are the closest equivalents to `dmesg`, `top` and `journalctl -f` for a cluster.

03. Form a hypothesis

A written hypothesis stops you flailing.

```
# Document in the incident ticket: 'Suspect probe regression after image bump'.
```

Expected: Hypothesis in the ticket.

Internals: Postmortems judge investigations on the *quality* of the hypothesis tree, not the time-to-fix.

04. Test and recover

Rollback is the cheapest test: if the previous revision works, the new one introduced the regression.

```
kubectl rollout history deploy/<svc>
kubectl rollout undo deploy/<svc>
kubectl rollout status deploy/<svc>
```

Expected: Service back to healthy.

Internals: Always recover first, root-cause second. Customers don't care about the why.

05. Validate

Validation = 'metrics agree AND no new errors AND a synthetic transaction succeeds'.

```
# Re-run the triage commands; confirm SLI dashboards back to green.
kubectl get pods -A --field-selector=status.phase!=Running | wc -l
```

Expected: Unhealthy pod count returns to baseline; Grafana SLI ≥ target.

Internals: Without explicit validation, you'll be paged again in 10 minutes.

06. Write the postmortem

Action items must be specific, owned and dated. Otherwise the incident repeats.

```
# Use the template: Timeline | Impact | Root cause | What went well | What didn't | Action items.
```

Expected: Document attached to the incident ticket.

Internals: Blameless postmortems are mandatory in mature SRE orgs; they convert outage cost into learning value.

Common Errors

Tunnel vision — Cause: Chasing the first hypothesis past the point of evidence · Fix: Time-box hypotheses to 15 minutes; if no progress, regroup

Root-cause before recovery — Cause: Engineer's instinct · Fix: Stop the bleeding first

Troubleshooting

• Always have a written hypothesis

• Always have an explicit recovery action

• Always validate before declaring resolution

Enterprise Relevance

This loop is the difference between a senior SRE and a button-pusher. It scales from a 1-pod misconfig to a multi-region outage.

Validation

• Service restored to baseline SLI

• Postmortem published within 24h

• At least one preventive action item filed

Related Incidents & Guide Sections

Incidents: K8S-INC-3001, K8S-INC-3002, K8S-INC-3003, K8S-INC-3004, K8S-INC-3005

• Troubleshooting methodology

• Enterprise patterns

Questions

Beginner

Q. What's the first goal of an incident response?

A. Stop customer impact (recover), not find root cause.

Intermediate

Q. Why time-box hypotheses?

A. Avoid tunnel vision; force re-evaluation against fresh evidence.

Advanced

Q. Why are blameless postmortems more valuable than blame ones?

A. They surface systemic issues; blame just hides them.

Scenario

Q. 30% of pods OOMKill after a deploy. You can't reach the dev who pushed it. Action?

A. Rollback the deploy; root-cause and contact dev after restoration.

Viva

Q. Walk me through your last (or hypothetical) P1.

A. Use the loop: triage → hypothesis → test → recover → validate → postmortem.